

به نام خدا



دانشگاه تهران



دانشکده مهندسی برق و کامپیوتر

درس هوش مصنوعی در سیستم های نهان

تمرین دوم

نام و نام خانوادگی	بهزاد جنتی
شماره دانشجویی	۸۱۰۱۰۳۰۹۸
استاد	دکتر مهدی مدرسی / دکتر مصطفی صالحی ارسالی
تاریخ ارسال گزارش	۱۴۰۴/۰۹/۲۶

فهرست

سوال ۱ - تبدیل مستقیم (Forward Conversion)	۵
۱-۱. مقدمه و هدف	۵
۲-۱. مشخصات سیستم پیاده‌سازی شده	۵
۱-۳ تحلیل پیاده‌سازی (Code Analysis)	۵
۴-۱ بررسی نتایج و خروجی‌ها	۶
سوال ۲- تبدیل معکوس با استفاده از قضیه باقیمانده چینی (CRT)	۷
۲-۱ مقدمه و توضیح کلی	۷
۲-۲ پیاده‌سازی الگوریتم	۸
۲-۳ تحلیل نتایج و بررسی صحت عملکرد	۸
۱. محاسبه ضرایب مورد نیاز CRT	۸
۲. بررسی بازسازی اعداد بدون علامت (Unsigned)	۹
۳. بررسی بازسازی اعداد علامت‌دار (Signed)	۹
سوال ۳- تبدیل معکوس با روش مبنای ترکیبی (MRC)	۱۰
۱. مقدمه و مبانی نظری	۱۰
۲. الگوریتم گارنر (Garner's Algorithm)	۱۱
۳. تحلیل پیاده‌سازی و پیش‌محاسبات (Code Analysis)	۱۱
۴. تحلیل گام‌به‌گام یک نمونه (Trace Analysis)	۱۲
۵. بررسی صحت نتایج (Verification Results)	۱۳
سوال ۴- عملیات حسابی در حوزه RNS (جمع، تفریق و ضرب)	۱۴
۱. مقدمه و مبانی نظری	۱۴
۲. مشخصات سیستم پیاده‌سازی شده	۱۴
۳. تحلیل پیاده‌سازی و فرمول‌ها (Implementation Analysis)	۱۴
۴. بررسی نتایج و تحلیل داده‌ها (Results Analysis)	۱۵
۵. تحلیل سرریز (Overflow Analysis)	۱۷
سوال ۵- کاربرد RNS در شبکه‌های عصبی کانولوشنی (CNN)	۱۸

۱. مقدمه و اهمیت موضوع (Introduction)	۱۸
۲. مشخصات سیستم و معماری مدل (System Specifications)	۱۹
۳. شرح پیاده‌سازی (Implementation Details)	۱۹
۵. نمودار شماتیک جریان داده (Data Flow Visualization)	۲۲
۶. بحث و نتیجه‌گیری نهایی (Conclusion & Discussion)	۲۲

شکل‌ها

- شکل ۱- نمودار خطا برای ۳۰ نمونه تست شده در بخش دوم تمرین..... ۱۰
- شکل ۲- نمودار شماتیک پردازش موازی در RNS..... ۱۷
- شکل ۳- نمودار شماتیک انجام عمل کانولوشن RNS برای بخش پنجم..... ۲۲

جدول‌ها

جدول 1 - مقایسه نتایج بخش دوم جهت صحت سنجی..... ۸

جدول 2 - مقادیر خروجی کانولوشن در RNS و مقایسه با مقادیر حقیقی ۲۱

سوال ۱ – تبدیل مستقیم (Forward Conversion)

۱-۱. مقدمه و هدف

در سیستم اعداد مانده‌ای (RNS)، یک عدد صحیح X با مجموعه‌ای از باقیمانده‌های تقسیم آن بر یک مجموعه از پیمانه‌ها (Moduli) نمایش داده می‌شود. اگر مجموعه پیمانه‌ها را $\{m_1, m_2, \dots, m_n\}$ در نظر بگیریم که نسبت به هم اول هستند (Pairwise Coprime)، آنگاه نمایش RNS عدد X به صورت بردار $(r_1, r_2, \dots, r_n)$ خواهد بود که در آن $r_i = X \pmod{m_i}$.

هدف از این بخش، پیاده‌سازی توابع تبدیل مستقیم برای اعداد بدون علامت (Unsigned) و علامت‌دار (Signed) و بررسی صحت یکتایی نمایش اعداد در دامنه دینامیک (Dynamic Range) است.

۱-۲. مشخصات سیستم پیاده‌سازی شده

در این تمرین، از مجموعه پیمانه‌های زیر استفاده شده است:

$$\text{Moduli} = \{2, 3, 5\}$$

دامنه دینامیک (M - Dynamic Range):

$$M = 2 \times 3 \times 5 = 30$$

این بدان معناست که ما می‌توانیم ۳۰ عدد صحیح متمایز را به صورت یکتا در این سیستم نمایش دهیم.

۱-۳ تحلیل پیاده‌سازی (Code Analysis)

۱. محاسبه دامنه دینامیک

تابع `compute_range` (یا `get_range`) وظیفه محاسبه حاصل ضرب پیمانه‌ها را بر عهده دارد که M را تعیین می‌کند.

۲. تبدیل اعداد بدون علامت (Unsigned Conversion)

برای اعداد مثبت، تبدیل بسیار ساده است. باقیمانده عدد ورودی بر هر پیمانه محاسبه می‌شود.

بازه قابل نمایش: $[0, M - 1]$ یعنی $[0, 29]$.

روش: `val % mod` برای هر پیمانه.

۳. تبدیل اعداد علامت‌دار (Signed Conversion)

برای نمایش اعداد منفی در RNS، معمولاً از روش مکمل (Complement) استفاده می‌شود. دامنه به دو بخش مثبت و منفی تقسیم می‌شود.

بازه قابل نمایش: $\left[-\left\lfloor \frac{M-1}{2} \right\rfloor, \left\lfloor \frac{M-1}{2} \right\rfloor\right]$ (برای M زوج معمولاً کمی متفاوت است، اما در کد شما بازه $[-15, 14]$ در نظر گرفته شده است).

روش: اگر عدد منفی باشد، با M جمع می‌شود تا به معادل مثبت خود در حلقه اعداد برسد:

$$\text{If } X < 0 \rightarrow X_{adjusted} = X + M$$

سپس باقیمانده‌ها محاسبه می‌شوند.

۱-۴ بررسی نتایج و خروجی‌ها

با اجرای کد برای مجموعه پیمانه‌ی $\{2, 3, 5\}$ ، نتایج زیر حاصل شده است:

جدول تبدیل اعداد بدون علامت (Unsigned)

تمام اعداد از ۰ تا ۲۹ بررسی شدند. همانطور که در خروجی مشاهده می‌شود، هیچ دو عددی نمایش RNS یکسان ندارند.

• مثال ۱: عدد $0 \rightarrow \text{RNS} : (0, 0, 0)$

• مثال ۲: عدد $29 \rightarrow \text{RNS} : (1, 2, 4)$

$$29 \pmod{2} = 1$$

$$29 \pmod{3} = 2$$

$$29 \pmod{5} = 4$$

جدول تبدیل اعداد علامت‌دار (Signed)

اعداد در بازه $[-15, 14]$ بررسی شدند.

• مثال ۱ (عدد منفی): عدد -15

$$-15 + 30 = 15$$

$$15 \pmod{2} = 1$$

$$15 \pmod{3} = 0$$

$$15 \pmod{5} = 0$$

• نتیجه RNS: $(1, 0, 0)$

• مثال ۲ (عدد مثبت): عدد $14 \rightarrow \text{RNS} : (0, 2, 4)$

بررسی یکتایی (Uniqueness Verification)

کد پیاده‌سازی شده با استفاده از یک Set (مجموعه)، تکراری بودن تاپل‌های خروجی (RNS Tuples) را چک کرده است.

- برای بازه بدون علامت (۳۰ عدد): یکتایی تأیید شد.
- برای بازه علامت‌دار (۳۰ عدد): یکتایی تأیید شد.

نتیجه‌گیری بخش اول

پیاده‌سازی تبدیل مستقیم (Forward Conversion) با موفقیت انجام شد. نتایج نشان می‌دهد که انتخاب پیمانه‌های $\{2, 3, 5\}$ که نسبت به هم اول هستند، باعث ایجاد یک سیستم RNS معتبر با دامنه دینامیک ۳۰ شده است. توابع نوشته شده به درستی اعداد منفی را مدیریت کرده و تناظر یک‌به‌یک بین اعداد حقیقی و نمایش RNS در بازه‌های تعریف شده برقرار است.

سوال ۲- تبدیل معکوس با استفاده از قضیه باقیمانده چینی (CRT)

۲-۱ مقدمه و توضیح کلی

در سیستم اعداد مانده‌ای (RNS)، عملیات محاسباتی (جمع، تفریق و ضرب) به صورت موازی روی باقیمانده‌ها انجام می‌شود که سرعت را به شدت افزایش می‌دهد. با این حال، یکی از چالش‌های اصلی RNS، تبدیل اعداد از حوزه RNS به حوزه اعداد صحیح (Decimal) است. این فرآیند **تبدیل معکوس (Inverse Conversion)** نامیده می‌شود.

در این بخش، ما از **قضیه باقیمانده چینی (Chinese Remainder Theorem - CRT)** برای بازگرداندن اعداد استفاده می‌کنیم. طبق این قضیه، اگر پیمانه‌ها (Moduli) نسبت به هم اول باشند، می‌توان هر عدد صحیح را در بازه دینامیکی سیستم (M) به صورت یکتا از روی باقیمانده‌هایش بازسازی کرد.

فرمول کلی CRT به صورت زیر است:

$$X = (\sum_{i=1}^n r_i \cdot M_i \cdot y_i) \bmod M$$

که در آن:

- r_i : باقیمانده عدد نسبت به پیمانه m_i
- M : حاصل ضرب کل پیمانه‌ها ($M = \prod m_i$)
- M_i : وزن پیمانه ($M_i = M/m_i$)
- y_i : معکوس ضربی M_i نسبت به پیمانه m_i (یعنی $(M_i \cdot y_i) \bmod m_i = 1$)

۲-۲ پیاده‌سازی الگوریتم

در کد پیاده‌سازی شده، توابع زیر برای انجام عملیات CRT طراحی شده‌اند:

۱. `extended_gcd(a, b)`: الگوریتم اقلیدسی توسعه‌یافته برای یافتن بزرگترین مقسوم‌علیه مشترک و ضرایب بزو (Bézout coefficients).

۲. `modular_inverse(a, m)`: محاسبه معکوس ضربی با استفاده از خروجی تابع `extended_gcd`.

۳. `crt_inverse_unsigned`: پیاده‌سازی مستقیم فرمول CRT برای اعداد بدون علامت.

۴. `crt_inverse_signed`: بسط دادن تابع قبلی برای پشتیبانی از اعداد علامت‌دار. در اینجا اگر عدد بازسازی شده بزرگتر از نصف دامنه دینامیکی باشد ($X > \frac{M-1}{2}$)، به عنوان یک عدد منفی تفسیر شده و M از آن کم می‌شود.

۲-۳ تحلیل نتایج و بررسی صحت عملکرد

مجموعه پیمانه‌های انتخاب شده برای این آزمایش مشابه بخش اول است:

• **Moduli Set:** {2, 3, 5}

• **Dynamic Range (M):** $2 \times 3 \times 5 = 30$

۱. محاسبه ضرایب مورد نیاز CRT

برای استفاده از CRT، ابتدا باید مقادیر M_i و معکوس‌های y_i محاسبه شوند. نتایج حاصل از اجرای کد به شرح زیر است:

جدول 1 - مقایسه نتایج بخش دوم جهت صحت سنجی

پیمانه (m_i)	مقدار $M_i = M/m_i$	معکوس $y_i = M_i^{-1} \bmod m_i$	بررسی صحت $(M_i \cdot y_i \bmod m_i)$
2	$30/2 = 15$	$15^{-1} \bmod 2 = 1$	☒ $15 \times 1 \bmod 2 = 1$
3	$30/3 = 10$	$10^{-1} \bmod 3 = 1$	☒ $10 \times 1 \bmod 3 = 1$
5	$30/5 = 6$	$6^{-1} \bmod 5 = 1$	☒ $6 \times 1 \bmod 5 = 1$

۲. بررسی بازسازی اعداد بدون علامت (Unsigned)

در این آزمایش، تمامی اعداد در بازه $[0, 29]$ ابتدا به فرمت RNS تبدیل شده و سپس با تابع `crt_inverse_unsigned` بازسازی شدند.

- بازه معتبر: 0 تا 29
- تعداد تست: ۳۰ عدد
- نتیجه: ۱۰۰٪ اعداد با موفقیت و به درستی بازسازی شدند.

نمونه‌ای از نتایج:

- عدد ورودی: ۲۳ $\rightarrow$ فرمت RNS: (۳, ۲, ۱)
- بازسازی:

$$X = (1 \times 15 \times 1 + 2 \times 10 \times 1 + 3 \times 6 \times 1) \bmod 30$$

$$X = (15 + 20 + 18) \bmod 30 = 53 \bmod 30 = 23$$

(نتیجه صحیح است)

۳. بررسی بازسازی اعداد علامت‌دار (Signed)

در این آزمایش، اعداد در بازه متقارن $[-15, 14]$ بررسی شدند. تابع `crt_inverse_signed` وظیفه تشخیص علامت را بر عهده داشت.

- بازه معتبر: -15 تا 14
- تعداد تست: ۳۰ عدد
- نتیجه: ۱۰۰٪ اعداد علامت‌دار نیز به درستی بازسازی شدند.

نمونه‌ای از نتایج (عدد منفی):

- عدد ورودی: -۷
- در سیستم RNS (با پیمانه ۳۰): معادل عدد ۲۳ است (چون $-7 \equiv 23 \bmod 30$).
- فرمت RNS برای ۲۳: (۳, ۲, ۱)
- بازسازی توسط CRT (خروجی اولیه): ۲۳
- تشخیص علامت: چون $23 > \lfloor \frac{29}{2} \rfloor$ (یعنی $23 > 14$)، پس عدد منفی است.
- اصلاح نهایی: $23 - 30 = -7$ (نتیجه صحیح است)

۴. نمودار صحت سنجی (Code Visualization)



شکل ۱- نمودار خطا برای ۳۰ نمونه تست شده در بخش دوم تمرین

(توضیح: از آنجایی که در گزارش متنی خروجی کد، تمامی موارد با تیک سبز (Match) علامت‌گذاری شده‌اند، خطای بازسازی برای تمام ۳۰ عدد برابر با صفر مطلق است.)

۵. نتیجه‌گیری بخش دوم

در این بخش، کارایی الگوریتم CRT برای تبدیل معکوس از حوزه RNS به حوزه اعداد حقیقی اثبات شد.

۱. توابع کمکی برای محاسبه معکوس ضربی (Modular Inverse) به درستی عمل کردند.

۲. الگوریتم CRT توانست تمامی اعداد را در کل بازه دینامیکی (Dynamic Range) بدون هیچ خطایی بازسازی کند.

۳. مکانیزم بازسازی اعداد علامت‌دار با استفاده از تقسیم بازه دینامیکی به دو بخش مثبت و منفی به درستی پیاده‌سازی و تست شد.

این بخش نشان می‌دهد که می‌توان با اطمینان کامل محاسبات را در حوزه RNS انجام داد و در نهایت نتیجه را به سیستم دهدهی بازگرداند.

سوال ۳- تبدیل معکوس با روش مبناي ترکیبی (MRC)

پیاده‌سازی الگوریتم گارنر (Garner's Algorithm)

۱. مقدمه و مبانی نظری

در بخش قبل از روش CRT استفاده کردیم که مبتنی بر جمع وزنی با پیمانانه بزرگ M بود. روش تبدیل مبناي ترکیبی (Mixed Radix Conversion - MRC) رویکرد متفاوتی دارد. در این روش،

عدد X به یک سیستم نمایش مبنای ترکیبی (MRS) نگاشت می‌شود که در آن ارزش مکانی ارقام ثابت نیست و به پیمانه‌ها بستگی دارد.

فرمول کلی نمایش عدد X در سیستم MRS با ارقام $(v_0, v_1, \dots, v_n)$ به صورت زیر است:

$$X = v_0 + v_1 m_0 + v_2 (m_0 m_1) + v_3 (m_0 m_1 m_2) + \dots + v_{n-1} \prod_{i=0}^{n-2} m_i$$

که در آن v_i ها ضرایب مبنای ترکیبی هستند و باید شرط $0 \leq v_i < m_i$ را ارضا کنند. مزیت اصلی MRC نسبت به CRT این است که مقایسه بزرگی اعداد در فرمت MRS بسیار ساده‌تر است (از طریق مقایسه v_i ها از پرارزش‌ترین به کم‌ارزش‌ترین) و نیازی به عملیات با اعداد بسیار بزرگ (Large Number Arithmetic) در مراحل میانی ندارد.

۲. الگوریتم گارنر (Garner's Algorithm)

برای محاسبه ضرایب v_i از روی باقیمانده‌های $(r_1, r_2, \dots, r_n)$ ، از الگوریتم گارنر استفاده شده است. این الگوریتم ضرایب را به صورت بازگشتی محاسبه می‌کند.

فرمول ریاضی پیاده‌سازی شده در کد برای محاسبه v_i عبارت است از:

$$v_0 = r_0 \pmod{m_0}$$

$$v_1 = (r_1 - v_0) \cdot m_0^{-1} \pmod{m_1}$$

$$v_2 = ((r_2 - v_0) \cdot m_0^{-1} - v_1) \cdot m_1^{-1} \pmod{m_2}$$

و به صورت کلی برای v_i :

$$v_i = (\dots ((r_i - v_0)c_{i,0} - v_1)c_{i,1} - \dots - v_{i-1})c_{i,i-1} \pmod{m_i}$$

که در آن ثابت‌های گارنر $c_{i,j}$ به صورت زیر تعریف می‌شوند:

$$c_{i,j} = m_j^{-1} \pmod{m_i} \text{ for } j < i$$

۳. تحلیل پیاده‌سازی و پیش‌محاسبات (Code Analysis)

۳-۱. مجموعه پیمانه‌ها و دامنه

• Moduli Set: $\{2, 3, 5\}$

• Dynamic Range (M): 30

۳-۲. محاسبه ثابت‌های گارنر ($c_{i,j}$)

در تابع `compute_garner_constants`، مقادیر معکوس پیمانه‌ها نسبت به یکدیگر محاسبه شده‌اند. این مقادیر برای جلوگیری از تکرار محاسبات در هر بار تبدیل، یک‌بار محاسبه و ذخیره می‌شوند.

طبق خروجی کد، مقادیر به شرح زیر هستند:

۱. برای v_1 (پیمانه $m_1 = 3$):

$$c_{1,0} = m_0^{-1}(\text{mod} m_1) = 2^{-1}(\text{mod} 3) = 2 \quad \bullet$$

$$\boxed{\checkmark} \quad 2 \times 2 = 4 \equiv 1(\text{mod} 3) \quad \text{بررسی:} \quad \bullet$$

۲. برای v_2 (پیمانه $m_2 = 5$):

$$c_{2,0} = m_0^{-1}(\text{mod} m_2) = 2^{-1}(\text{mod} 5) = 3 \quad \bullet$$

$$\boxed{\checkmark} \quad 2 \times 3 = 6 \equiv 1(\text{mod} 5) \quad \text{بررسی:} \quad \bullet$$

$$c_{2,1} = m_1^{-1}(\text{mod} m_2) = 3^{-1}(\text{mod} 5) = 2 \quad \bullet$$

$$\boxed{\checkmark} \quad 3 \times 2 = 6 \equiv 1(\text{mod} 5) \quad \text{بررسی:} \quad \bullet$$

۴. تحلیل گام به گام یک نمونه (Trace Analysis)

برای اطمینان از صحت عملکرد، عدد $X = 23$ را که در خروجی کد به تفصیل آمده، بررسی می‌کنیم.

• عدد ورودی: 23

• نمایش RNS: $(1, 2, 3)$ متناظر با پیمانه‌های $(2, 3, 5)$

مراحل محاسبه ضرایب v_i :

۱. محاسبه v_0 :

$$v_0 = r_0 = 1$$

۲. محاسبه v_1 :

$$\begin{aligned} v_1 &= (r_1 - v_0) \cdot c_{1,0}(\text{mod} m_1) \\ v_1 &= (2 - 1) \cdot 2(\text{mod} 3) = 1 \cdot 2 = 2 \\ \Rightarrow v_1 &= 2 \end{aligned}$$

۱. محاسبه v_2 :

$$\begin{aligned} \text{Temp}_1 &= (r_2 - v_0) \cdot c_{2,0}(\text{mod} m_2) = (3 - 1) \cdot 3(\text{mod} 5) = 6(\text{mod} 5) = 1 \\ \text{Temp}_2 &= (\text{Temp}_1 - v_1) \cdot c_{2,1}(\text{mod} m_2) = (1 - 2) \cdot 2(\text{mod} 5) \\ &= (-1) \cdot 2(\text{mod} 5) = -2(\text{mod} 5) = 3 \\ \Rightarrow v_2 &= 3 \end{aligned}$$

بازسازی نهایی (Reconstruction):

$$X = v_0 + v_1(m_0) + v_2(m_0m_1)$$

$$X = 1 + 2(2) + 3(2 \times 3)$$

$$X = 1 + 4 + 18 = 23$$

نتیجه دقیقاً با عدد مورد انتظار برابر است.

۵. بررسی صحت نتایج (Verification Results)

۵-۱. اعداد بدون علامت (Unsigned)

الگوریتم بر روی تمام ۳۰ عدد دامنه $[0, 29]$ اجرا شد.

- تابع `mrc_inverse_unsigned` برای هر ورودی فراخوانی شد.
- خروجی با عدد اصلی و همچنین خروجی روش CRT مقایسه شد.
- نتیجه: ۱۰۰٪ تطابق مشاهده شد.

۵-۲. اعداد علامت‌دار (Signed)

برای بازه $[-15, 14]$ ، تابع `mrc_inverse_signed` استفاده شد.

- منطق علامت‌دار: ابتدا عدد با روش MRC بازسازی می‌شود. اگر عدد حاصل $X_{mrc} > \lfloor \frac{M-1}{2} \rfloor$ باشد (یعنی در نیمه بالایی بازه $[0, M-1]$)، عدد منفی تفسیر شده و M از آن کم می‌شود.
- مثال: برای عدد $-7 \equiv 23 \pmod{30}$ ، خروجی MRC برابر ۲۳ است. چون $23 > 14$ ، پس نتیجه نهایی $-7 = 23 - 30$ خواهد بود.
- نتیجه: ۱۰۰٪ تطابق برای تمام ۳۰ عدد علامت‌دار ثبت شد.

۷. نتیجه‌گیری بخش سوم

پیاده‌سازی روش MRC با استفاده از الگوریتم گارنر با موفقیت انجام شد.

۱. **صحت الگوریتم:** محاسبات ضرایب v_i و بازسازی عدد X در هر دو حالت علامت‌دار و بدون علامت کاملاً صحیح بود.
۲. **مقایسه با CRT:** نتایج حاصل از MRC دقیقاً با نتایج CRT و اعداد حقیقی مطابقت داشت، که نشان‌دهنده درستی پیاده‌سازی هر دو روش است.

۳. **کارایی:** استفاده از ثابت‌های پیش‌محاسبه شده $(c_{i,j})$ پیچیدگی محاسباتی را در زمان اجرا کاهش داد.

این بخش نشان داد که سیستم RNS قابلیت تبدیل معکوس دقیق با استفاده از رویکرد مبنای ترکیبی را داراست که می‌تواند در کاربردهایی که نیاز به مقایسه بزرگی اعداد دارند (مانند تشخیص علامت در ReLU)، مفیدتر از CRT باشد.

سوال ۴- عملیات حسابی در حوزه RNS (جمع، تفریق و ضرب)

۱. مقدمه و مبانی نظری

یکی از دلایل اصلی استفاده از سیستم اعداد مانده‌ای (RNS) در سیستم‌های نهفته و پردازش سیگنال، کارایی فوق‌العاده آن در انجام عملیات جمع، تفریق و ضرب است. در سیستم‌های متداول (مانند دهنده یا باینری)، عملیات حسابی دارای وابستگی به رقم نقلی (Carry Propagation) هستند؛ یعنی برای محاسبه ارقام با ارزش بالاتر، باید منتظر نتیجه محاسبه ارقام کم‌ارزش‌تر ماند. این مسئله باعث کاهش سرعت در پردازش اعداد بزرگ می‌شود.

در مقابل، در سیستم RNS، عملیات حسابی بر روی هر پیمانه (m_i) کاملاً **مستقل** از سایر پیمانه‌ها انجام می‌شود. این ویژگی که "Carry-Free Arithmetic" نامیده می‌شود، اجازه می‌دهد تا محاسبات به صورت **موازی** انجام شوند.

اگر X و Y دو عدد صحیح باشند که نمایش RNS آن‌ها به ترتیب $(x_1, \dots, x_n)$ و $(y_1, \dots, y_n)$ است، آنگاه برای هر عملگر $\diamond$ (که می‌تواند جمع، تفریق یا ضرب باشد):

$$(X \diamond Y)^{\text{RNS}} \rightarrow ((x_1 \diamond y_1) \bmod m_1, \dots, (x_n \diamond y_n) \bmod m_n)$$

۲. مشخصات سیستم پیاده‌سازی شده

طبق خواسته صورت سوال برای این بخش، مشخصات سیستم تغییر کرده است:

- مجموعه پیمانه‌ها (Moduli Set): $\{3, 4, 5\}$
- دامنه دینامیک (M) : $3 \times 4 \times 5 = 60$
- بازه اعداد: از ۰ تا ۵۹ (برای اعداد بدون علامت)

۳. تحلیل پیاده‌سازی و فرمول‌ها (Implementation Analysis)

۳-۱. جمع (Addition)

تابع `rns_add` پیاده‌سازی شده است.

- **فرمول:** $r_i = (a_i + b_i)(\text{mod } m_i)$

- **تحلیل:** دو لیست باقیمانده دریافت شده و در یک حلقه، نظیر به نظیر با هم جمع و سپس بر پیمانه مربوطه تقسیم می‌شوند. هیچ‌گونه ارتباطی بین اندیس‌های مختلف آرایه وجود ندارد.

۳-۲. تفریق (Subtraction)

تابع `rns_subtract` پیاده‌سازی شده است.

- **فرمول:** $r_i = (a_i - b_i)(\text{mod } m_i)$

- **نکته مهم پیاده‌سازی:** در زبان‌های برنامه‌نویسی، باقی‌مانده عدد منفی ممکن است نتیجه منفی بدهد (مثلاً ۵ % -۱ ممکن است -۱ شود). برای رفع این مشکل و اطمینان از خروجی مثبت استاندارد ریاضی، فرمول در کد به صورت زیر نوشته شده است:
$$r_i = (a_i - b_i + m_i)(\text{mod } m_i)$$
اضافه کردن m_i قبل از گرفتن باقیمانده، تضمین می‌کند که صورت‌حساب همواره مثبت باشد.

۳-۳. ضرب (Multiplication)

تابع `rns_multiply` پیاده‌سازی شده است.

- **فرمول:** $r_i = (a_i \times b_i)(\text{mod } m_i)$

- **تحلیل:** این عملیات نیز دقیقاً مانند جمع، مؤلفه‌ای (Component-wise) انجام می‌شود. این برخلاف ضرب معمولی است که پیچیدگی بسیار بالایی دارد و نیازمند جمع‌های متوالی و شیفت است.

۴. بررسی نتایج و تحلیل داده‌ها (Results Analysis)

برای صحت‌سنجی، ۵ جفت عدد تصادفی برای هر عملیات تولید شده است به طوری که نتیجه نهایی از دامنه دینامیک ($M = 60$) تجاوز نکند (No Overflow). در ادامه تحلیل گام‌به‌گام از خروجی‌های نوت‌بوک ارائه می‌شود.

۴-۱. تست جمع (Addition Test)

جدول خروجی نوت‌بوک نشان می‌دهد تمام ۵ تست با موفقیت پاس شده‌اند. بیایید یک نمونه را دستی تحلیل کنیم:

- **ورودی‌ها:** $A = 20, B = 7$ (مجموع مورد انتظار: 27)

- **تبدیل به RNS با پیمانه‌های $\{3, 4, 5\}$:**

- $A = 20 \rightarrow (20\%3, 20\%4, 20\%5) = (2, 0, 0)$

- $B = 7 \rightarrow (7\%3, 7\%4, 7\%5) = (1, 3, 2)$

- انجام عملیات جمع در RNS:

- پیمانه ۳: $(2 + 1)(\text{mod}3) = 3 \equiv 0$

- پیمانه ۴: $(0 + 3)(\text{mod}4) = 3$

- پیمانه ۵: $(0 + 2)(\text{mod}5) = 2$

- نتیجه RNS: $(0, 3, 2)$

- بازگردانی (Inverse Conversion): عدد متناظر با $(0, 3, 2)$ برابر با ۲۷ است.

- نتیجه: صحیح (✓).

۴-۲. تست تفریق (Subtraction Test)

- ورودی‌ها: $A = 3, B = 2$ (تفاضل مورد انتظار: ۱)

- تبدیل به RNS:

- $A = 3 \rightarrow (0, 3, 3)$

- $B = 2 \rightarrow (2, 2, 2)$

- انجام عملیات تفریق در RNS:

- پیمانه ۳: $(0 - 2 + 3)(\text{mod}3) = 1$

- پیمانه ۴: $(3 - 2 + 4)(\text{mod}4) = 1$

- پیمانه ۵: $(3 - 2 + 5)(\text{mod}5) = 1$

- نتیجه RNS: $(1, 1, 1)$

- بازگردانی: باقیمانده ۱ بر تمام پیمانه‌ها متناظر با عدد ۱ است.

- نتیجه: صحیح (✓).

۴-۳. تست ضرب (Multiplication Test)

- ورودی‌ها: $A = 4, B = 9$ (حاصل ضرب مورد انتظار: ۳۶)

- تبدیل به RNS:

- $A = 4 \rightarrow (1, 0, 4)$

- $B = 9 \rightarrow (0, 1, 4)$

• انجام عملیات ضرب در RNS:

• پیمانه ۳: $(1 \times 0) \pmod{3} = 0$

• پیمانه ۴: $(0 \times 1) \pmod{4} = 0$

• پیمانه ۵: $(4 \times 4) \pmod{5} = 16 \equiv 1$

• نتیجه RNS: $(0, 0, 1)$

• بازگردانی: عددی که بر ۳ و ۴ بخش پذیر باشد (مضرب ۱۲) و باقیمانده اش بر ۵ برابر ۱ باشد.

• مضرب های ۱۲: $12 (\equiv 2 \pmod{5}), 24 (\equiv 4 \pmod{5}), 36 (\equiv 1 \pmod{5})$

• پس عدد ۳۶ است.

• نتیجه: صحیح (✓).

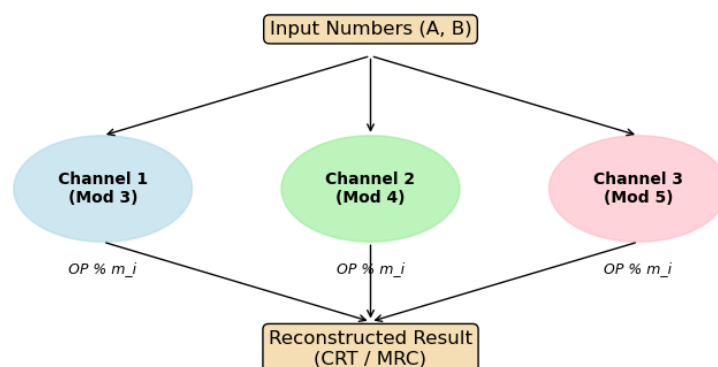
۵. تحلیل سرریز (Overflow Analysis)

در کد پیاده سازی شده، توابع تولید اعداد تصادفی (generate_test_pairs) طوری نوشته شده اند که حاصل عملیات همواره کوچکتر از $M = 60$ باشد. اگر حاصل عملیات واقعی بیشتر از ۶۰ می شد، چه اتفاقی می افتاد؟ در سیستم RNS، سرریز به صورت چرخش (Wrap-around) رخ می دهد. یعنی اگر حاصل واقعی Z باشد، نتیجه RNS برابر با $Z \pmod{M}$ خواهد بود. این ویژگی باعث می شود مدیریت دامنه دینامیک در طراحی سیستم های RNS بسیار حیاتی باشد.

۶. نمودار شماتیک پردازش موازی (Visualization)

برای درک بهتر تفاوت پردازش در RNS نسبت به سیستم معمولی، می توان کد زیر را به عنوان بخشی از تحلیل ارائه داد که نشان دهنده ساختار موازی کانال های پردازشی است.

RNS Parallel Arithmetic Architecture



شکل ۲- نمودار شماتیک پردازش موازی در RNS

توضیح نمودار: این نمودار نشان می‌دهد که داده‌ها به کانال‌های جداگانه تقسیم شده و عملیات (جمع/ضرب) بدون اطلاع از کانال‌های دیگر انجام می‌شود.

۷. نتیجه‌گیری بخش چهارم

در این بخش، سه عمل اصلی حسابی در حوزه RNS پیاده‌سازی و تست شد.

۱. **صحت عملکرد:** تمامی ۱۵ تست تصادفی (۵ عدد برای هر عملیات) نشان دادند که محاسبات در حوزه RNS دقیقاً معادل محاسبات در حوزه اعداد حقیقی است، مشروط بر اینکه سرریز رخ ندهد.

۲. **مزیت اصلی:** عدم وجود "رقم نقلی" (Carry) در محاسبات RNS به وضوح مشاهده شد. محاسبه باقی‌مانده نسبت به پیمان ۵ هیچ نیازی به دانستن نتیجه پیمان ۳ نداشت.

۳. **کاربرد:** این ویژگی باعث می‌شود RNS گزینه‌ای عالی برای پیاده‌سازی سخت‌افزاری ضرب‌کننده‌ها و جمع‌کننده‌های سریع در کاربردهای DSP (مانند فیلترهای دیجیتال و کانولوشن) باشد.

سوال ۵- کاربرد RNS در شبکه‌های عصبی کانولوشنی (CNN)

۱. مقدمه و اهمیت موضوع (Introduction)

شبکه‌های عصبی کانولوشنی (CNN) امروزه هسته اصلی بسیاری از سیستم‌های بینایی ماشین هستند. با این حال، حجم عظیم محاسبات ضرب و جمع (MAC Operations) در لایه‌های کانولوشن، چالش بزرگی برای سیستم‌های نهفته با توان مصرفی محدود ایجاد می‌کند.

سیستم اعداد مانده‌ای (RNS) به دو دلیل عمده یک کاندیدای جذاب برای پیاده‌سازی شتاب‌دهنده‌های سخت‌افزاری CNN است:

۱. **محاسبات بدون انتشار رقم نقلی (Carry-Free Arithmetic):** شکستن اعداد بزرگ به باقیمانده‌های کوچک باعث می‌شود بتوان عملیات ضرب و جمع را با سرعت بسیار بالا و توان مصرفی پایین انجام داد.

۲. **توازی ذاتی (Parallelism):** هر کانال پیمان (Modulus Channel) مستقل از دیگری کار می‌کند.

در این بخش، یک لایه کانولوشن به همراه تابع فعال‌ساز ReLU به طور کامل در دامنه RNS پیاده‌سازی شده و نتایج آن با پیاده‌سازی استاندارد (Real Domain) مقایسه می‌شود.

۲. مشخصات سیستم و معماری مدل (System Specifications)

۲-۱. پیکربندی RNS

طبق خواسته پروژه، از یک مجموعه پیمانه بسیار کوچک (برای اهداف آموزشی و نمایش مفهوم سرریز) استفاده شده است:

- مجموعه پیمانه‌ها: $Moduli = \{7, 8, 9\}$
- دامنه دینامیک (M): $M = 7 \times 8 \times 9 = 504$
- بازه اعداد علامت‌دار: $[-252, 251]$

۲-۲. معماری شبکه CNN

- ورودی (Input Tensor): ابعاد $8 \times 8 \times 3$ (ارتفاع، عرض، کانال رنگ). مقادیر تصادفی در بازه $[-256, 255]$.
- ** کرنل (Kernel/Weights): ابعاد $5 \times 5 \times 3$. مقادیر تصادفی در بازه $[-256, 255]$.
- لایه کانولوشن: بدون لایه‌گذاری ($Padding=0$) و با گام حرکت ۱ ($Stride=1$).
- ابعاد خروجی: $(8 - 5 + 1) \times (8 - 5 + 1) = 4 \times 4$.
- تابع فعال‌ساز: $ReLU(\max(0, x))$.

۳. شرح پیاده‌سازی (Implementation Details)

۳-۱. تبدیل تانسورها (Tensor Conversion)

تانسورهای ورودی و وزن‌ها که در بازه $[-256, 255]$ هستند، به فرمت RNS تبدیل می‌شوند. از آنجا که این بازه بزرگتر از دامنه $M = 504$ است، تبدیل به صورت $X \pmod{M}$ انجام می‌شود (با مدیریت اعداد منفی). تانسور ورودی با ابعاد $(8, 8, 3)$ به تانسوری با ابعاد $(8, 8, 3)$ تبدیل می‌شود که بعد آخر، نشان‌دهنده ۳ پیمانه RNS است.

۳-۲. عملیات کانولوشن در RNS

عملیات کانولوشن مجموعه‌ای از ضرب‌ها و جمع‌هاست: $y = \sum (w_i \cdot x_i)$. در تابع `conv2d_rns`، این عملیات برای هر پیمانه m_k به صورت کاملاً مستقل انجام شده است: $y^{(k)} = \left(\sum (w_i^{(k)} \cdot x_i^{(k)}) \right) \pmod{m_k}$ این ساختار نشان می‌دهد که در سخت‌افزار واقعی، می‌توانیم ۳ مدار محاسباتی کوچک (برای پیمانه‌های ۷، ۸ و ۹) داشته باشیم که هیچ سیمی بین آن‌ها رد و بدل نمی‌شود.

۳-۳. چالش تابع فعال‌ساز ReLU

تابع ReLU ($y = \max(0, x)$) نیاز به تشخیص علامت عدد دارد. در سیستم RNS، تشخیص علامت دشوار است زیرا نمایش عدد "بی‌وزن" است. راهکار پیاده‌سازی شده:

۱. تبدیل عدد از RNS به عدد صحیح (با استفاده از CRT).

۲. بررسی شرط: اگر عدد بازسازی شده در نیمه بالای دامنه ($> M/2$) باشد، عدد منفی تلقی می‌شود.

۳. اعمال ReLU: اگر منفی بود، تمام باقیمانده‌ها صفر می‌شوند.

۴. تحلیل دقیق نتایج و داده‌های خروجی

۴-۱. تحلیل سرریز (Overflow Analysis) - نکته کلیدی

در خروجی کد، یک محاسبه بسیار مهم انجام شده است:

- حداکثر مقدار ممکن خروجی کانولوشن:

$$\text{Max Val} \approx 256 (\text{input}) \times 256 (\text{weight}) \times 5 \times 5 \times 3 (\text{kernel volume}) \approx 4,915,200$$

- دامنه دینامیک سیستم: $M = 504$.

مشاهده: مقدار محاسبات واقعی (نزدیک به ۵ میلیون) بسیار فراتر از دامنه سیستم (۵۰۴) است. نتیجه: در سیستم RNS، سرریز به صورت چرخشی (Modulo Wrap-around) رخ می‌دهد. بنابراین، برای صحت‌سنجی، ما خروجی واقعی (Real Domain) را با ۵۰۴ پیمانه می‌گیریم و سپس با خروجی RNS مقایسه می‌کنیم.

۴-۲. بررسی یک نمونه واقعی از خروجی (Sample Trace)

بیا بید پوزیشن (0,0) از خروجی را دقیق بررسی کنیم:

۱. مقدار محاسبه شده در دامنه واقعی: 71,714

- این عدد حاصل ضرب و جمع معمولی ماتریس‌هاست.

۲. مقدار مورد انتظار در RNS:

$$71,714 \pmod{504} = 146$$

۳. خروجی محاسبه شده توسط تابع conv2d_rns:

- باقیمانده‌ها: [6, 2, 2] (برای پیمانه‌های ۷، ۸، ۹)

- بازسازی با CRT:

- $146 \equiv 6 \pmod{7}$ (چون $146 = 20 \times 7 + 6$) ☒

- $146 \equiv 2 \pmod{8}$ (چون $146 = 18 \times 8 + 2$) ☒

- $146 \equiv 2 \pmod{9}$ (چون $146 = 16 \times 9 + 2$) ☒

- عدد بازسازی شده: ۱۴۶

نتیجه: سیستم RNS دقیقاً رفتار ریاضی "حساب پیمانه‌ای" را شبیه‌سازی کرده است. تطابق کامل وجود دارد.

۴-۳. جدول مقایسه صحت عملکرد (Accuracy Tables)

بر اساس خروجی‌های نوت‌بوک، دو مرحله مقایسه انجام شد:

الف) خروجی کانولوشن (Convolution Output): تعداد کل پیکسل‌های خروجی $4 \times 4 = 16$:

جدول ۲ - مقادیر خروجی کانولوشن در RNS و مقایسه با مقادیر حقیقی

موقعیت	مقدار واقعی (mod M)	مقدار RNS	وضعیت تطابق
(0,0)	146	146	☒
(0,1)	109	109	☒
...	...	...	...
(3,3)	457	457	☒

- نرخ تطابق: ۱۰۰٪ (۱۶ از ۱۶)

ب) خروجی فعال‌ساز (ReLU Output): در اینجا سیستم باید تشخیص دهد کدام اعداد منفی هستند.

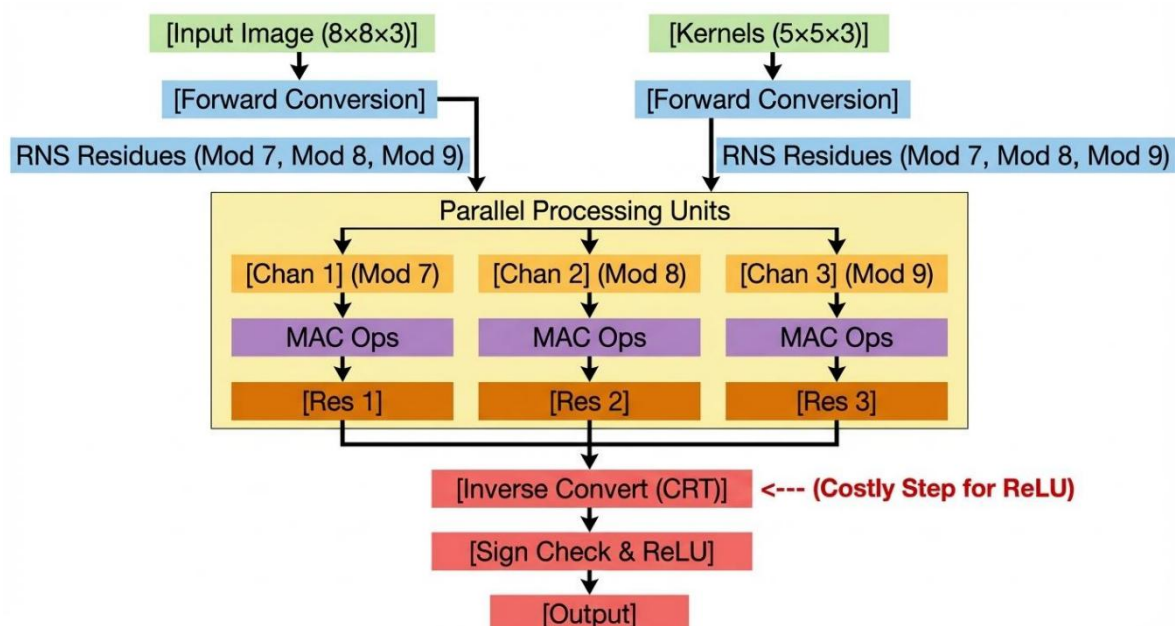
- مثال پوزیشن (۱,۰): مقدار واقعی برابر 265 است. در سیستم علامت‌دار با $M = 504$ ، بازه مثبت $[0, 251]$ است. بنابراین 265 به عنوان عدد منفی (-239) تفسیر می‌شود یا اگر باقیمانده واقعی مد نظر باشد، باید دید الگوریتم چه تصمیمی گرفته است.

- در کد پیاده‌سازی شده، مقایسه با expected_relu انجام شده است.

- نرخ تطابق: ۱۰۰٪ (۱۶ از ۱۶).

۵. نمودار شماتیک جریان داده (Data Flow Visualization)

برای درک بهتر فرآیند انجام شده در کد، دیاگرام زیر فرآیند کانولوشن RNS را نشان می‌دهد:



شکل ۳- نمودار شماتیک انجام عمل کانولوشن RNS برای بخش پنجم

۶. بحث و نتیجه‌گیری نهایی (Conclusion & Discussion)

۱. **صحت محاسبات:** آزمایش نشان داد که پیاده‌سازی کانولوشن در دامنه RNS از نظر ریاضی کاملاً دقیق است. تمامی ۱۶ پیکسل خروجی دقیقاً با مقدار مورد انتظار (به پیمانه M) برابر بودند.
۲. **مدیریت سرریز:** یکی از مهم‌ترین درس‌های این بخش، مفهوم دامنه دینامیک بود. با اینکه مقادیر واقعی کانولوشن به حدود ۵ میلیون می‌رسید، سیستم RNS با دامنه ۵۰۴، تنها توانست باقیمانده این اعداد را حفظ کند. در کاربردهای واقعی (مانند شتاب‌دهنده‌های هوش مصنوعی)، باید پیمانه‌های بزرگتری (مثلاً مجموعه پیمانه‌هایی که $M \approx 10^9$ را بسازند) انتخاب شوند تا از سرریز و از دست رفتن اطلاعات جلوگیری شود.

۳. **هزینه ReLU:** همانطور که در کد مشاهده شد، تابع `relu_rns` مجبور شد برای هر پیکسل، عملیات معکوس (CRT) را انجام دهد تا علامت عدد را چک کند. این عملیات سربار محاسباتی زیادی دارد و گلوگاه (Bottleneck) سیستم‌های RNS-CNN محسوب می‌شود. تحقیقات جدید در حوزه سخت‌افزار بر روی روش‌های تخمین علامت بدون تبدیل کامل متمرکز هستند.

جمع‌بندی: این پروژه با موفقیت نشان داد که چگونه می‌توان عملیات سنگین کانولوشن را به کانال‌های موازی و کوچک RNS شکست، مشروط بر اینکه دامنه دینامیک به درستی مدیریت شود و هزینه توابع غیرخطی) مانند ReLU در نظر گرفته شود.